RollNo:-18
Name:Hemant Sachin Gavali
Msc ICT-3

Open Source Web Development - 304

Practical Assignment-2

Question 1

Practical Assignment: Blog Management REST API with Authentication (Node.js, Express,

MongoDB, EJS)

Scenario: A startup wants to build the backend and a basic server-rendered interface for a personal

blogging platform where registered users can sign up, log in, and manage their own blog posts. Only the

author of a post (or an admin) should be able to edit or delete it. You have been asked to build this

system.

Build an application using Node.js, Express.js, MongoDB, Mongoose, and EJS (for rendering views) that

implements the following:

1. User Authentication

● POST /api/auth/register – Register a new user (name, email, password) with the password

hashed using bcrypt.

● POST /api/auth/login – Authenticate a user and return a JWT token.

● GET /api/auth/profile – A protected route that returns the logged-in user's profile.

2. Blog Post CRUD API (protected routes, linked to the logged-in author)

● POST /api/posts – Create a new blog post (title, content, tags, published status).

● GET /api/posts – Retrieve all blog posts belonging to the logged-in user.

● GET /api/posts/:id – Retrieve a single blog post by ID.

● PUT /api/posts/:id – Update a blog post by ID (only the author or an admin can update).

● DELETE /api/posts/:id – Delete a blog post by ID (only the author or an admin can delete).

● POST /api/posts/:id/image – Upload or replace a featured image for a blog post (multipart/form-

data) using Multer.

3. EJS Views

● Render an EJS page (views/posts.ejs) that displays all published blog posts in a styled list.

● Render an EJS page (views/login.ejs) with a login form that submits to your login route.

● Render an EJS page (views/dashboard.ejs, protected) showing the logged-in user's own posts

with edit/delete links, visible only after login.

4. File Uploading

● Use Multer to handle multipart/form-data uploads for a blog post’s featured image and/or a

user’s profile picture.

● Restrict uploads to image MIME types (jpeg, png, webp) and enforce a maximum file size (e.g.,

2MB); reject and return a clear error for anything else.

● Store uploaded files in a local /uploads directory (or a cloud service such as Cloudinary/AWS S3)

and save only the resulting file path/URL on the Post or User document.

● Serve uploaded images as static assets and render them in views/posts.ejs and

views/dashboard.ejs.

● Delete the associated image file from storage when a post (or its image) is deleted or replaced.

5. Security

● Use Helmet to set secure HTTP headers, and configure CORS to only allow trusted origins.

● Apply rate limiting (e.g., express-rate-limit) on /api/auth/login and /api/auth/register to mitigate

brute-force and credential-stuffing attacks.

● Validate and sanitize all request input (e.g., with express-validator or Joi) to guard against NoSQL

injection and malformed payloads.

● Keep secrets (JWT signing key, MongoDB URI, upload credentials) in environment variables via

dotenv and never commit them to source control (.env.example only).

● Set a reasonable JWT expiry, sign tokens with a strong secret, and reject/refresh appropriately

on expiry.

● Validate uploaded file types on the server (not just by extension) and never trust the client-

supplied filename when saving to disk.

6. Requirements

● Use bcrypt to hash and compare passwords — never store plain text passwords.

● Use JWT middleware to protect all post-related API and dashboard routes.

● Use Mongoose schemas to model User and Post, with each post referencing its author via

ObjectId and each user having a role field (user/admin).

● Ensure a user can only view/edit/delete their own posts, unless they are an admin.

● Return appropriate HTTP status codes and consistent JSON responses for the API routes.

● Never trust client input for authorization decisions — always re-check ownership/role server-

side, even if the UI hides edit/delete links.

● Apply the same Security requirements above (Helmet, rate limiting, input validation, env-based

secrets) across both the API and file-upload routes.

Deliverable: A working Node.js/Express project with proper folder structure (models, routes,

RollNo:-18
Name:Hemant Sachin Gavali
Msc ICT-3

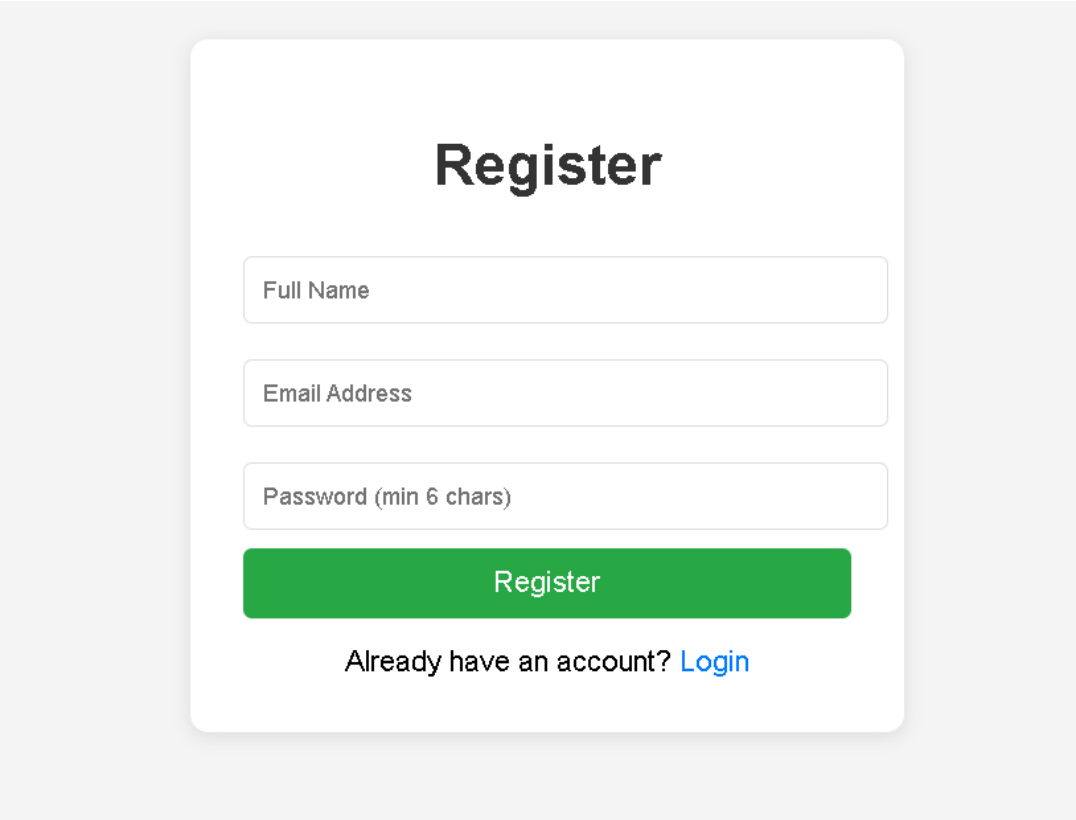
controllers, middleware, views, uploads), a README.md explaining setup and usage (including a

.env.example listing required environment variables), and example API requests (curl or Postman)

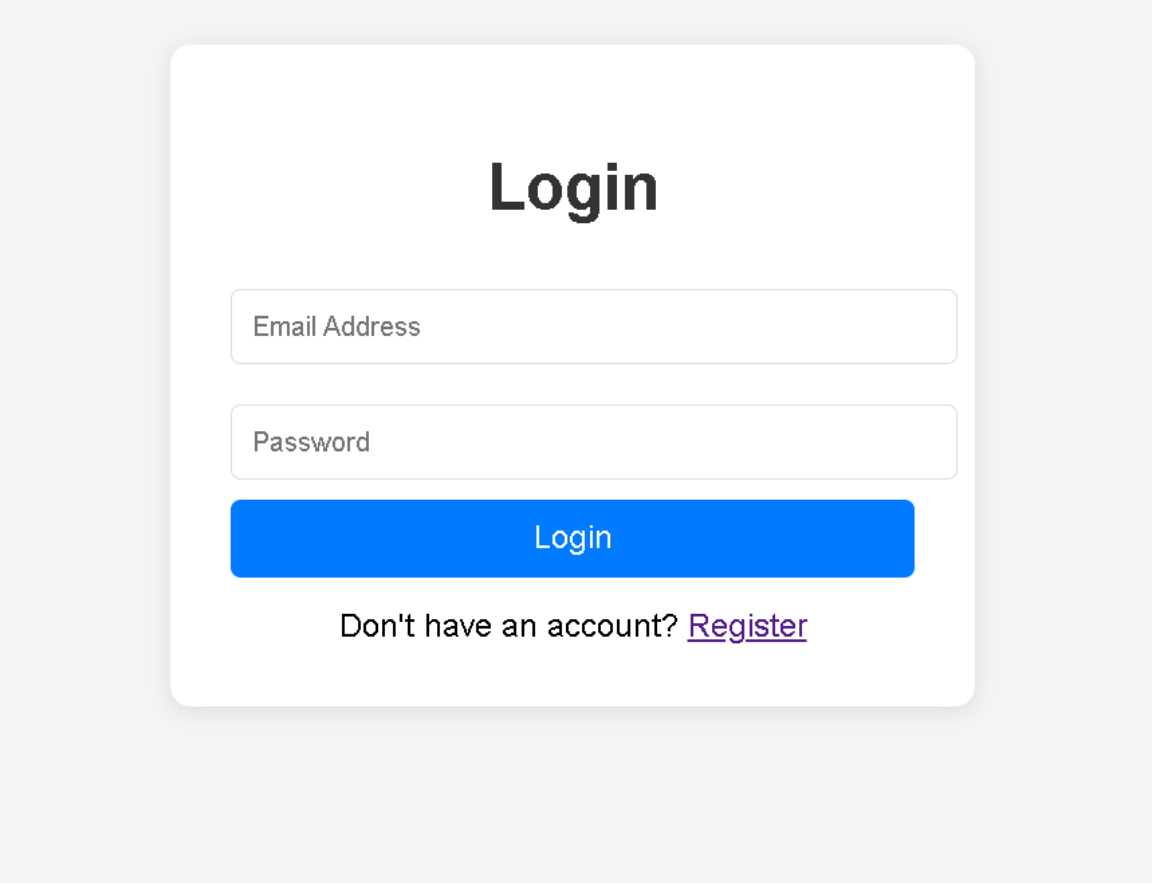
demonstrating registration, login, all blog post CRUD operations, and image upload, plus screenshots or

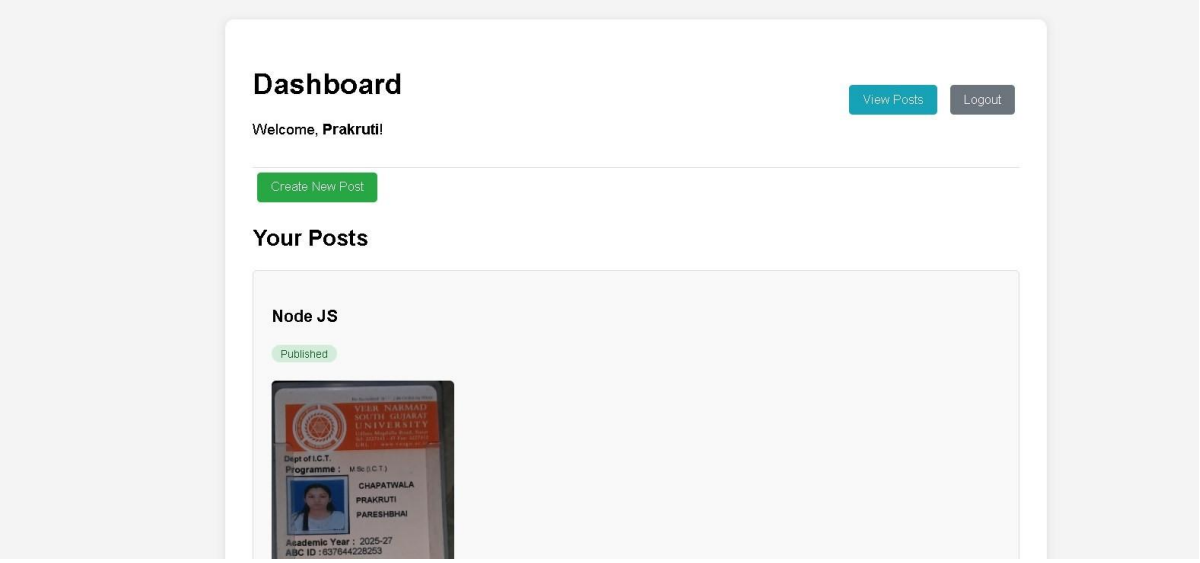
a demo of the EJS pages.

Output:-

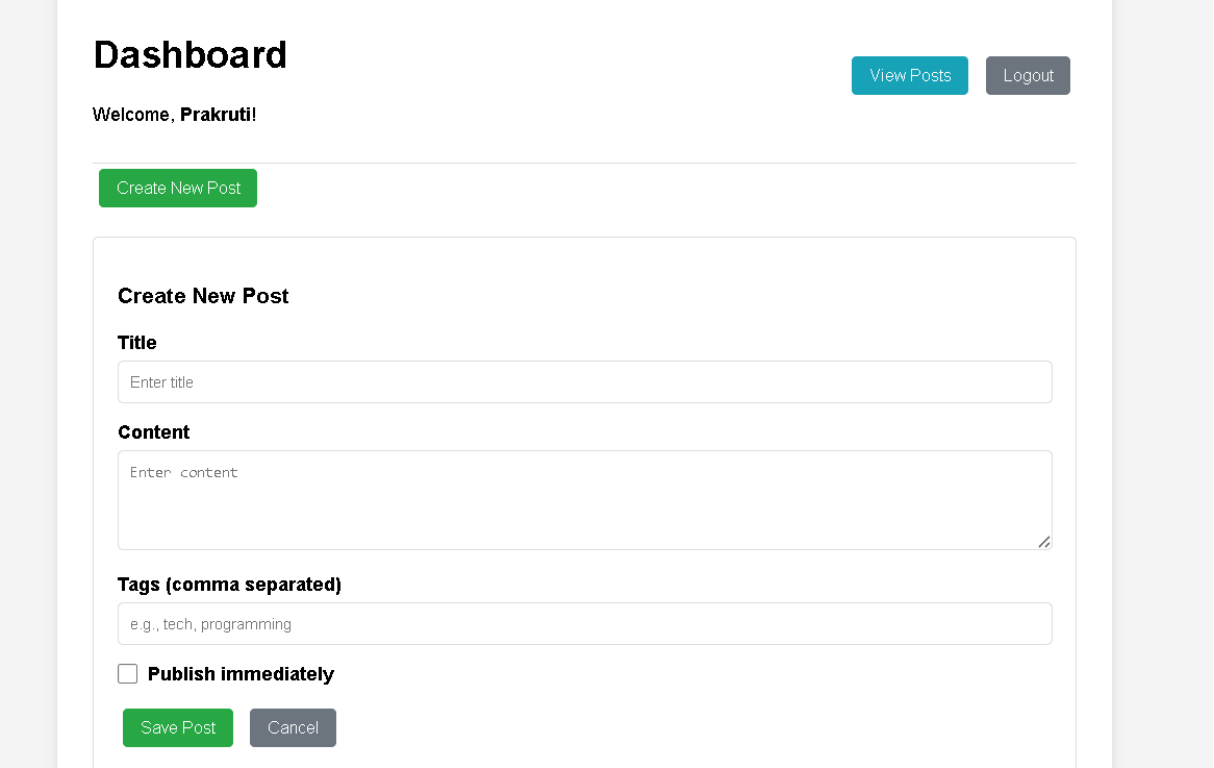


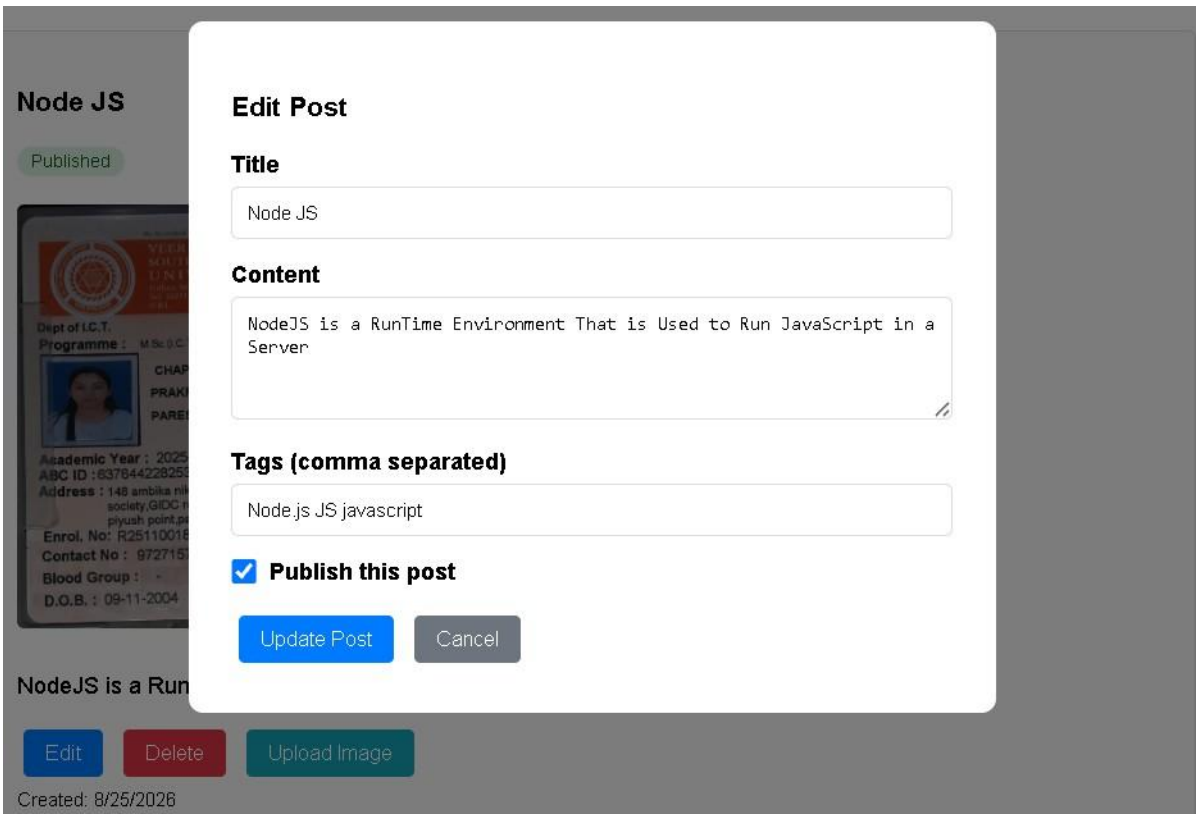
RollNo:-18
Name:Hemant Sachin Gavali
Msc ICT-3



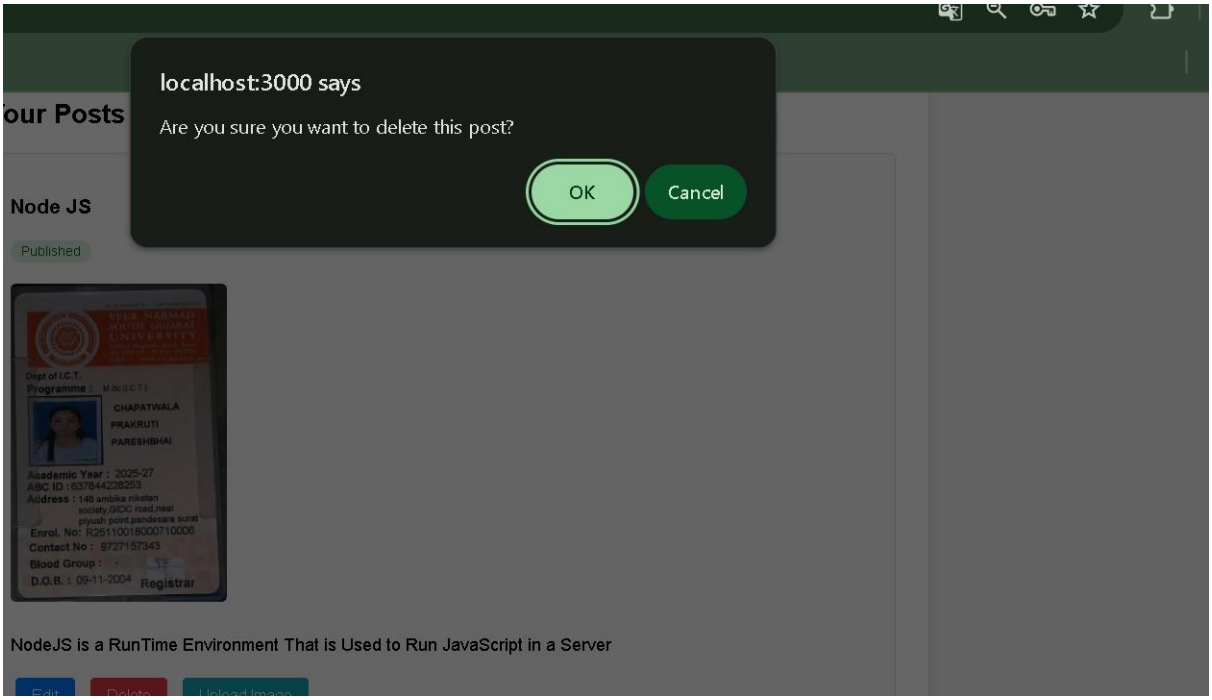


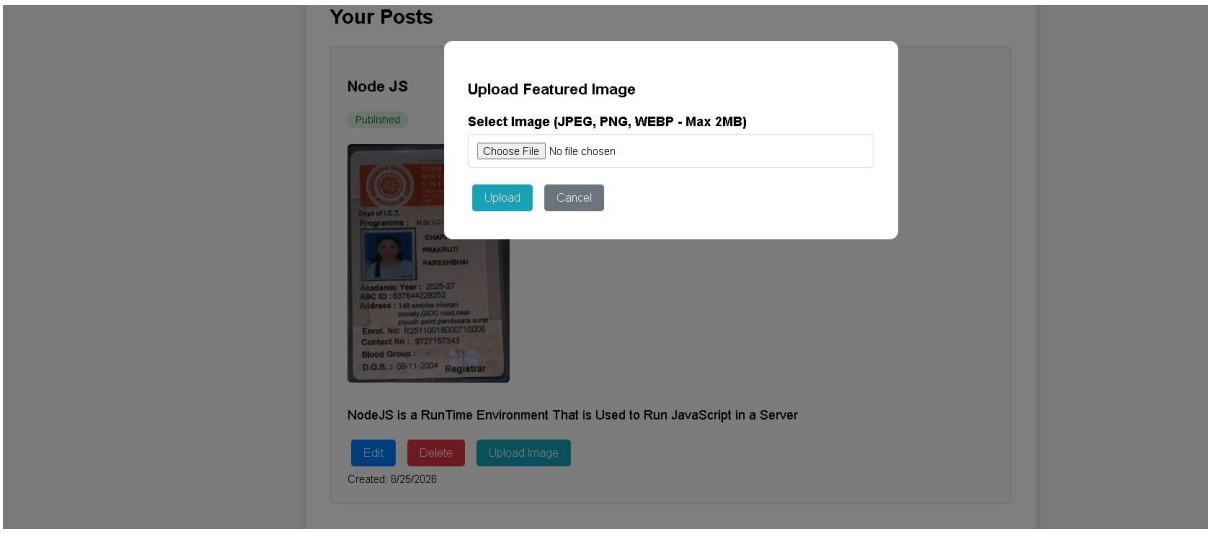
RollNo:-18
Name:Hemant Sachin Gavali
Msc ICT-3





RollNo:-18
Name:Hemant Sachin Gavali
Msc ICT-3





Question 2

Secure Signup for Bookstore Customers

Scenario: A new customer is creating an account on the online bookstore to start placing orders for

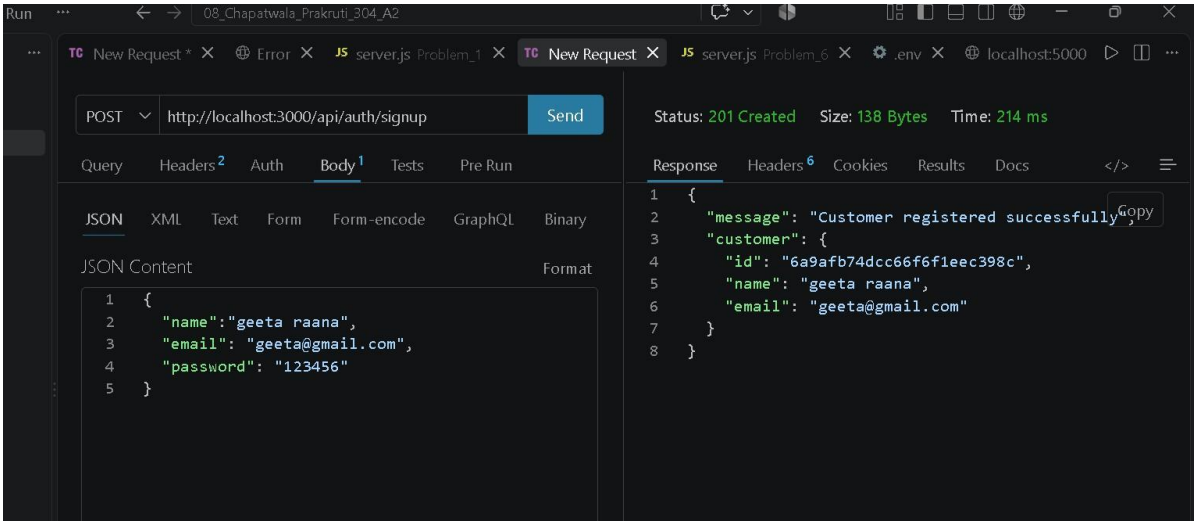
books.

Task: Write a program that hashes the customer's password using bcrypt before storing it in MongoDB,

and write the corresponding logic to verify the password when the same customer logs in later.

Output:-

RollNo:-18
Name:Hemant Sachin Gavali
Msc ICT-3



Question 3

Issuing a Digital Membership Token on Login

Scenario: Once a customer logs in successfully, the store must issue them a token (their digital

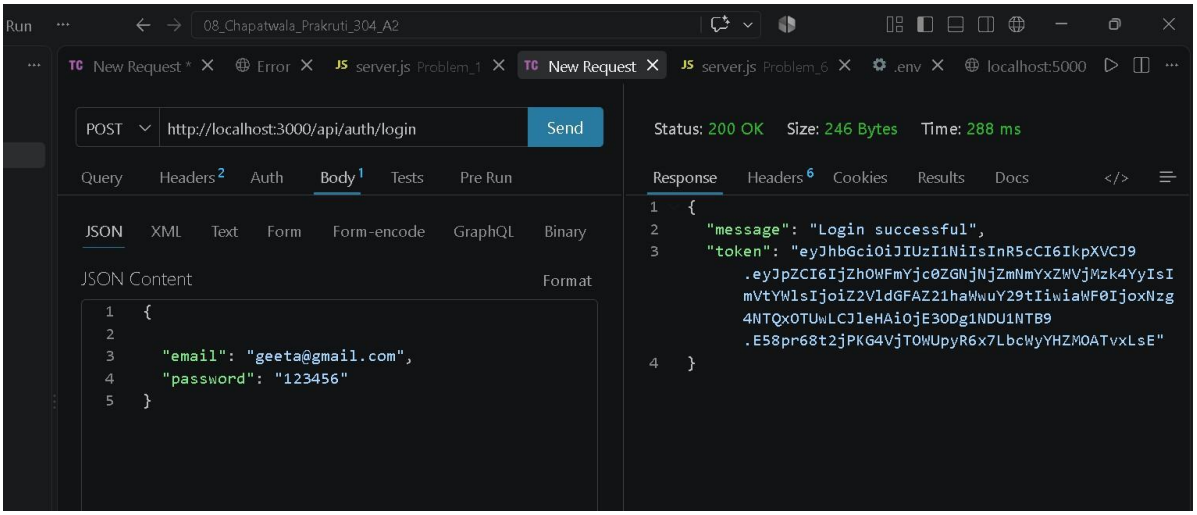
membership pass) so they don't need to log in again for every request, such as viewing their orders or

checking out.

Task: Write a program to generate a JWT token on successful login, and a middleware to verify this

token before allowing access to any protected route (e.g., placing an order or viewing order history).

Output:-



Question 4

RollNo:-18
Name:Hemant Sachin Gavali
Msc ICT-3

Defining Customer and Admin Roles

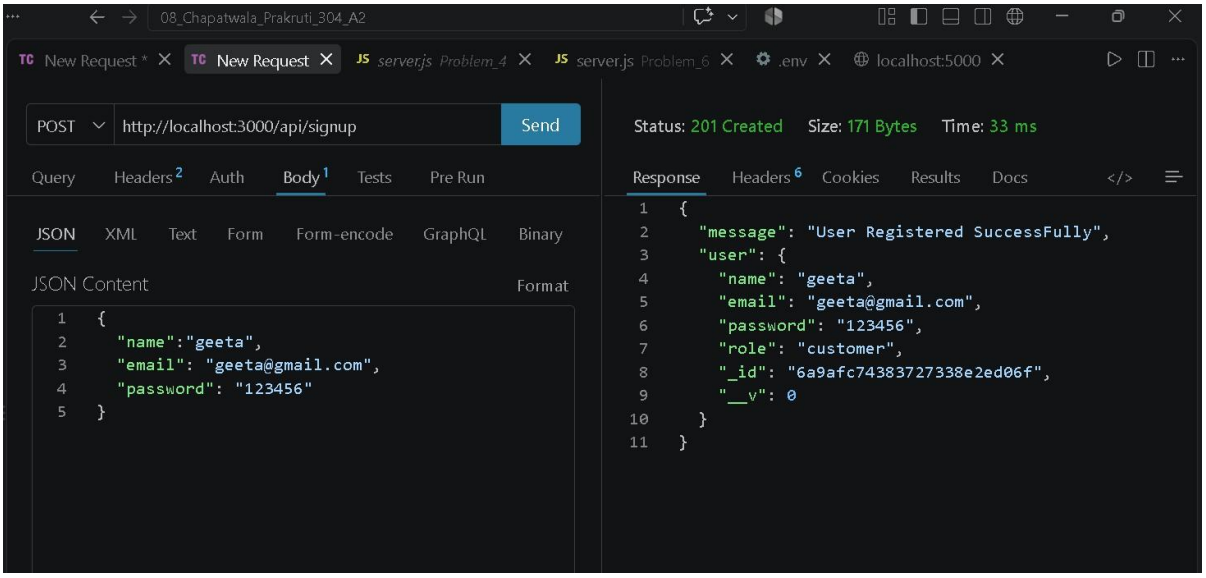
Scenario: The bookstore platform has two kinds of accounts — regular Customers who browse and buy

books, and Admins who manage inventory, pricing, and all customer orders.

Task: Write a Mongoose User schema with a role field ('customer' / 'admin') that defaults to 'customer',

so every new signup is treated as a regular customer unless explicitly promoted to admin.

Output-



Question 5

Admin-Only Access to the Inventory Dashboard

Scenario: The bookstore has an inventory management route (GET /api/admin/inventory) that lists stock

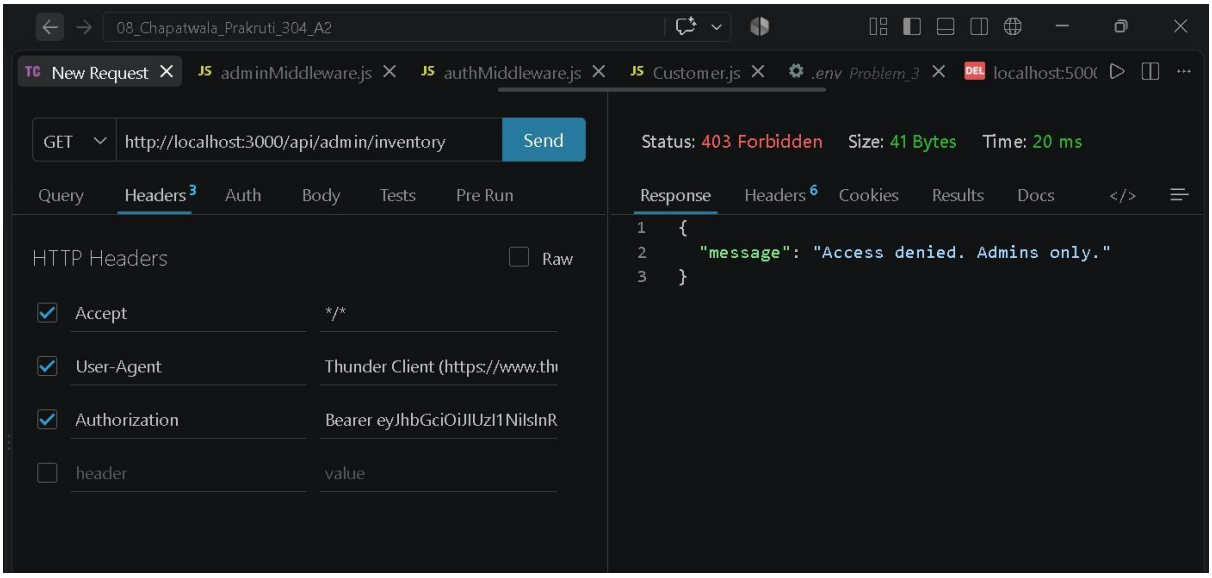
levels and sales data for every book, meant only for store admins.

Task: Write an authorization middleware that checks the logged-in user's role and returns 403

Forbidden if the user is not an 'admin', preventing regular customers from accessing this route.

Output:-

RollNo:-18
Name:Hemant Sachin Gavali
Msc ICT-3



Question 6

Cancelling an Order — Customer or Admin Only

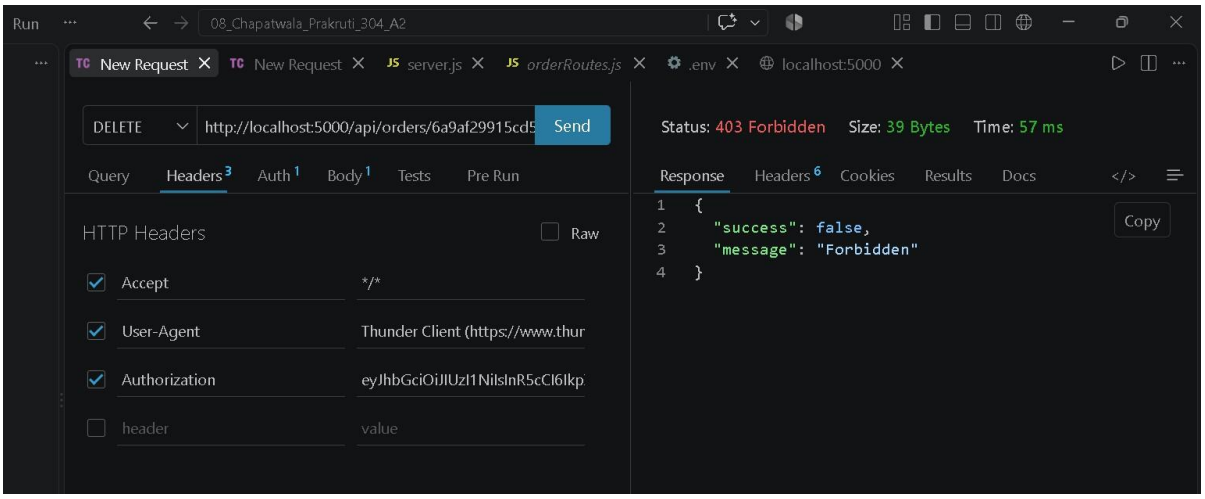
Scenario: A customer should be able to cancel their own order before it ships. Separately, an admin

should be able to cancel any customer's order — for example, when a book goes out of stock.

Task: Write a DELETE /api/orders/:id handler that allows an order to be cancelled only if the logged-in

user is the customer who placed the order, or holds the admin role, and returns 403 Forbidden

otherwise.



RollNo:-18
Name:Hemant Sachin Gavali
Msc ICT-3

